

Experiment-8

Aim:

- A. A company maintains customer information, product information, and sales order information in three separate tables: customer_master, product_catalog, and sales_orders.

The company wants to provide an order report to an external client. The report must contain the Order ID, Order Date, Customer Name, Product Name, and Final Amount.

- For every order, first calculate the Gross Amount using:
$$\text{Gross Amount} = \text{Unit Price} \times \text{Quantity}$$
- Then calculate the Discount Amount using:
$$\text{Discount Amount} = \text{Gross Amount} \times \text{Discount Percentage} / 100$$
- Finally, calculate the Final Amount using:
$$\text{Final Amount} = \text{Gross Amount} - \text{Discount Amount}$$
- Create a complex view using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.
- The company also wants to provide access to an external client. Create a PostgreSQL user named client_user and give this user only SELECT permission on the view. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

Tools / Platform Used:

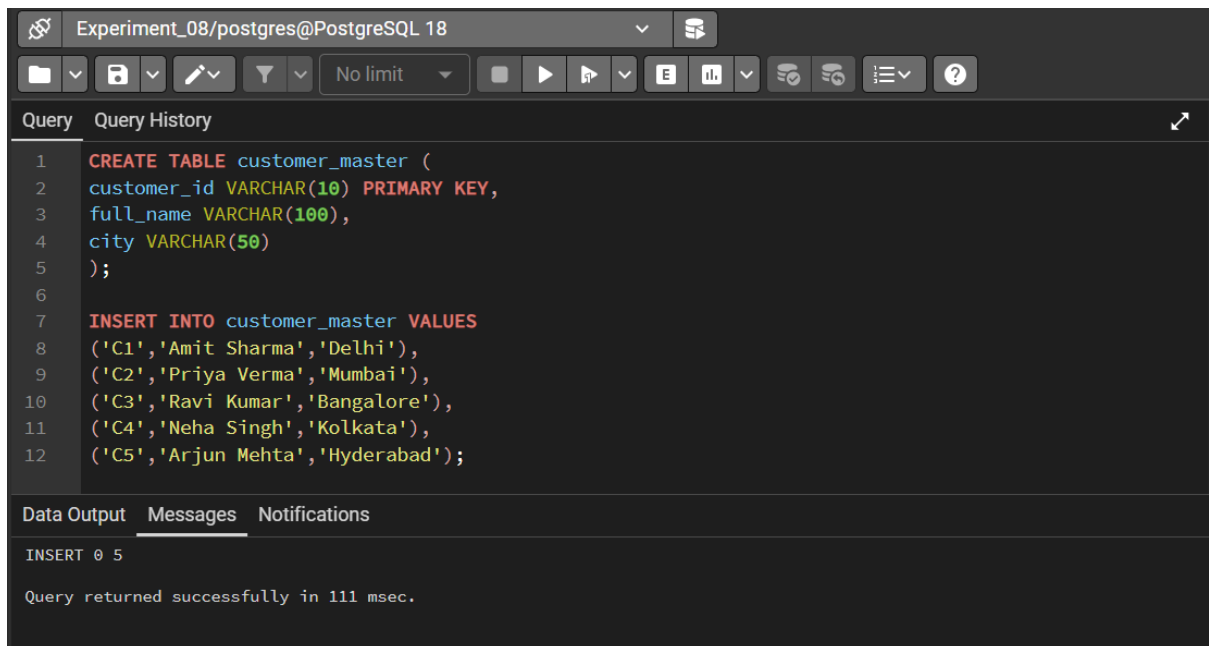
- Windows
- PostgreSQL
- PgAdmin

Theory:

- A Materialized View (MV) is a database object that stores the results of a query physically
- Unlike a normal view, the result is not calculated every time we query it.
- It is useful when a query is complex, expensive, and executed frequently

Code and Output:

A.

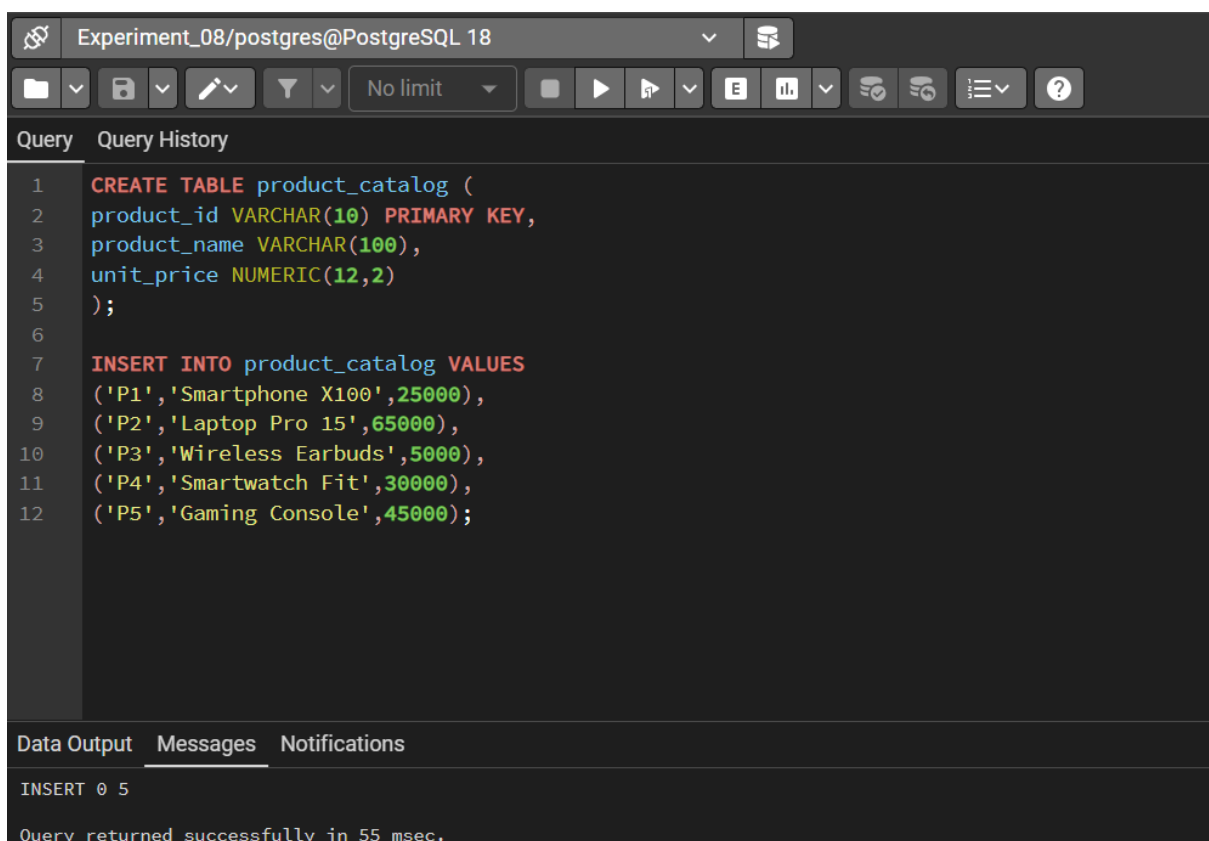


The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is 'Experiment_08/postgres@PostgreSQL 18'. The toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE customer_master (  
2   customer_id VARCHAR(10) PRIMARY KEY,  
3   full_name VARCHAR(100),  
4   city VARCHAR(50)  
5 );  
6  
7 INSERT INTO customer_master VALUES  
8 ('C1','Amit Sharma','Delhi'),  
9 ('C2','Priya Verma','Mumbai'),  
10 ('C3','Ravi Kumar','Bangalore'),  
11 ('C4','Neha Singh','Kolkata'),  
12 ('C5','Arjun Mehta','Hyderabad');
```

Below the query editor, the 'Data Output' tab is active, showing the results of the execution:

```
INSERT 0 5  
  
Query returned successfully in 111 msec.
```



The screenshot shows a PostgreSQL query editor interface. The title bar indicates the connection is 'Experiment_08/postgres@PostgreSQL 18'. The toolbar includes icons for file operations, query execution, and settings. The 'Query' tab is active, displaying the following SQL code:

```
1 CREATE TABLE product_catalog (  
2   product_id VARCHAR(10) PRIMARY KEY,  
3   product_name VARCHAR(100),  
4   unit_price NUMERIC(12,2)  
5 );  
6  
7 INSERT INTO product_catalog VALUES  
8 ('P1','Smartphone X100',25000),  
9 ('P2','Laptop Pro 15',65000),  
10 ('P3','Wireless Earbuds',5000),  
11 ('P4','Smartwatch Fit',30000),  
12 ('P5','Gaming Console',45000);
```

Below the query editor, the 'Data Output' tab is active, showing the results of the execution:

```
INSERT 0 5  
  
Query returned successfully in 55 msec.
```

Experiment_08/postgres@PostgreSQL 18

Query Query History

```
1 CREATE TABLE sales_orders (  
2 order_id INT PRIMARY KEY,  
3 product_id VARCHAR(10) REFERENCES product_catalog(product_id),  
4 quantity INT,  
5 customer_id VARCHAR(10) REFERENCES customer_master(customer_id),  
6 discount_percent NUMERIC(5,2),  
7 order_date DATE  
8 );  
9  
10 INSERT INTO sales_orders VALUES  
11 (1,'P1',2,'C1',5,'2025-09-01'),  
12 (2,'P2',1,'C2',10,'2025-09-02'),  
13 (3,'P3',3,'C3',0,'2025-09-03'),  
14 (4,'P4',1,'C4',8,'2025-09-04'),  
15 (5,'P5',1,'C5',12,'2025-09-05'),  
16 (6,'P1',1,'C2',5,'2025-09-06'),  
17 (7,'P3',2,'C1',0,'2025-09-07');
```

Data Output Messages Notifications

INSERT 0 7

Query returned successfully in 93 msec.

Experiment_08/postgres@PostgreSQL 18

Query Query History

```
1 CREATE OR REPLACE VIEW order_report AS  
2 SELECT  
3 so.order_id,  
4 so.order_date,  
5 cm.full_name AS customer_name,  
6 pc.product_name,  
7 ROUND(  
8 (pc.unit_price * so.quantity) -  
9 ((pc.unit_price * so.quantity) * so.discount_percent / 100), 2) AS final_amount  
10 FROM sales_orders AS so  
11 JOIN customer_master AS cm  
12 ON so.customer_id = cm.customer_id  
13 JOIN product_catalog AS pc  
14 ON so.product_id = pc.product_id;  
15  
16 SELECT * FROM order_report ORDER BY order_id;
```

Data Output Messages Notifications

Showing rows: 1 to 7

	order_id integer	order_date date	customer_name character varying (100)	product_name character varying (100)	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.00
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.00
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.00
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.00
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.00
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.00
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.00

Result:

The queries for the given tasks were written and executed successfully. The tasks given were completed successfully.

Learning Outcomes:

- Practiced PostgreSQL
- Got hands-on experience on working with materialized views in PostgreSQL.